

radosgw-admin UX and Documentation Improvements

Candidate: Rotem Shapira

Mentors: Yuval Lifshitz, Jacques Heunis

Repository: [radosgw-admin-ux-docs-improvements](#)

1. Introduction & Background

I am a Computer Science graduate from the Technion - Israel Institute of Technology. My involvement with Open Source and Ceph began recently under the mentorship of Yuval Lifshitz, focusing on performance optimizations within the RADOS Gateway (RGW). Specifically, I have been working on replacing the `rgw_map` (an `std::unordered_map` protected by a `table_mutex`) with Intel's TBB `concurrent_hash_map` to enable fine-grained locking and reduce contention in high-concurrency scenarios.

During this work, I identified a bug in the RGW Lua background thread where iterations were failing unexpectedly. My mentor diagnosed that reusing the same Lua VM across iterations caused stale state to persist, leading to the iteration failures. I implemented and submitted the fix ([PR #67396](#)) ensuring a fresh VM is created for each iteration, which has been merged into the Ceph main branch. This experience provided me with hands-on exposure to Ceph's internal architecture, its multi-threaded environment, and its build system.

I am applying for this project because, through my investigation of the RGW codebase for this proposal, I realized the significant "pain points" caused by undocumented or mismatched CLI commands. I believe my background in C++ and my existing familiarity with the Ceph development workflow will allow me to provide a robust, long-term solution to these documentation issues, while continuing to contribute to and learn from a large-scale storage system.

2. Analysis of Existing Mismatches

Currently, radosgw-admin documentation is a manual, error-prone process involving four disconnected sources:

1. The source code (radosgw-admin.cc)
2. The --help (-h) output
3. The man page (doc/man/8/radosgw-admin.rst)
4. The admin guide (doc/radosgw/admin.rst)

Summary of Documentation Mismatches

I conducted a thorough audit comparing the radosgw-admin --help output with the Man Page and Admin Guide. The results indicates that a significant portion of the CLI surface is either undocumented or inconsistently documented and highlights the need for automated validation between CLI definitions and documentation:

Summary Table

Category	Total	Missing in man page	Missing in -h	Missing in admin.rst
Commands	~220	68	2	190
Flags / Parameters	~160	46	2	128

Note: Totals were computed by combining commands and flags from --help output with additional commands discovered in source code and other documentation sources.

Command Mismatches

Issue Type	Count	Examples
Present in -h but missing in man page	68	user policy attach, account create, mfa create, dedup stats
Present in man page but missing in -h	2	object manifest, role modify
Present in -h but missing from admin guide (admin.rst)	188	(see full list of only 30 existing commands here)

Flags / Parameters Mismatches

Issue Type	Count	Examples
Present in -h but missing in man page	46	--account-id, --max-users, --ratelimit-scope
Present in man page but missing in -h	2	--subscription, --event-id
Completely undocumented	1	--group-id
Present in -h but missing from admin guide (admin.rst)	125	(see full list of only 32 existing flags here)

Key Observations

- **Admin guide coverage is very limited:**
admin.rst documents only ~30/220 commands (~14%) and 32/160 flags (~20%), leaving most CLI functionality undocumented.
The guide does not fully reflect the current CLI surface.
- **--help output is also incomplete:**
Although much more comprehensive than the admin guide, some commands and parameters (e.g., --group-id) are missing from -h, showing that the built-in help does not fully cover all functionality.
- **Entire feature groups undocumented:**
Commands such as dedup *, mfa * and account * exist in code and -h, but are missing from man page and admin guide.
- **Hidden parameters:**
--group-id exists in the code but is not documented in man page or -h.
- **Potential naming inconsistencies (needs validation):**
 - role-policy delete vs role-policy rm
 - role modify vs role-trust-policy modify

These may indicate aliases, renames, or semantic differences between commands.

Full List of Findings

A full list of all commands and parameters with mismatches is available here:

[radosgw-admin-audit.md](#)

3. Framework Evaluation

Project Needs

The core requirement for this project is to establish a single source of truth for the radosgw-admin CLI commands, subcommands, and options. The framework chosen must satisfy the following critical criteria:

1. Minimal dependencies and easy integration

- Header-only framework.
- Actively maintained and easy to include in Ceph build system.

2. Single source of truth + Automatic documentation generation

- All command definitions, descriptions, flags, and options must be defined in one place (C++ code) to prevent drift between CLI, man pages, and admin guide.
- Adding a new command or option updates the CLI, man page, and admin guide automatically.

3. Nested subcommands

- Support for multi-level command hierarchies (e.g., bucket logging flush) natively.

4. Tree introspection / exportability

- Ability to programmatically traverse and extract the full command tree (commands → subcommands → options, with descriptions, types, and required flags), enabling integration with external documentation generation tools to generate man pages and admin guide automatically.

5. Context-aware help generation

- Automatic --help per command and per subcommand.

6. Scoping and validation

- Options must appear only where relevant (per-command scoping).
- Required flags are enforced automatically per command.
- Hierarchical error messages when a subcommand or required argument is missing or invalid – where behavior changes are introduced, they will be done gradually (warnings before errors) to avoid breaking existing use-cases.

7. Global options with Ceph-compatible placement

- True global flags work at any command nesting level.
- Both before or after subcommand usage is supported.

8. Backward compatibility with existing CLI

- Existing commands and behaviors must continue to work. Output formatting, error messages, and option handling must remain consistent with legacy radosgw-admin.
- Behavior changes (such as stricter validation of duplicate flags or partial matches) are not core deliverables of this project. Where such improvements are introduced, they will follow a gradual migration path – emitting warnings first, with errors only after users have had time to adapt.

9. Documentation scope

- The primary focus is complete reference-style documentation – --help output and the man page (radosgw-admin.rst). The admin guide is a secondary goal: stable anchors generated per command will enable easy linking from the admin guide to relevant reference sections, without requiring full admin guide coverage as a deliverable.

Command-Line Parsing frameworks Examined

I evaluated several C++ CLI frameworks to find the best fit for Ceph's requirements and needs:

1. boost::program_options

Pros:

- Mature and widely used; already integrated in Ceph.
- Supports required arguments enforcement and backward compatibility.
- Actively maintained and stable.

Cons / Limitations:

- Nested subcommands (3+ levels) require manual workarounds.
- Auto-generated help requires manual setup; per-command help is not native.
- No built-in tree introspection.

Project Fit:

Boost is reliable and already used in Ceph, but its limitations on nested commands and help generation make it less suitable for building a fully hierarchical, user-friendly CLI for new features.

2. CLI11**Pros:**

- Natively supports deep nested subcommands (3+ levels), per-command help, hierarchical help suggestions, and full tree introspection (important for documentation).
- Header-only and easy to integrate.
- Actively maintained and modern C++17 features supported.
- Provides auto-generated help and per-command help out of the box.

Cons / Limitations:

- Requires C++17, which may be a consideration for backward compatibility in older codebases.

Project Fit:

CLI11 fits the project's requirements best. Its ability to handle complex command hierarchies and export command trees makes it ideal for our documentation needs. Since Ceph already supports C++17, this is a viable choice.

3. Lyra

Pros:

- Header-only, simple syntax.
- Provides basic validation capabilities.
- Actively maintained.

Cons / Limitations:

- Limited support for nested subcommands.
- Help generation is partial; hierarchical help suggestions are limited.
- Tree introspection not supported.

Project Fit:

Lyra could work for simple CLI tools but does not fully meet the project's needs for nested commands and tree introspection.

4. args

Pros:

- Header-only, lightweight.
- Required arguments enforcement supported.

Cons / Limitations:

- Nested subcommands are limited.
- Help generation is basic.
- Tree introspection not supported.

Project Fit:

Suitable for small, flat command-line interfaces but insufficient for a complex CLI with deep subcommands.

5. TCLAP

Pros:

- Stable, C++11-compatible, simple syntax, well-known from older projects.
- Required arguments enforcement supported.

Cons / Limitations:

- No support for nested subcommands beyond a flat structure.
- No hierarchical help suggestions or tree introspection.

Project Fit:

Does not satisfy the requirements for hierarchical CLI structures; limited usefulness for this project.

6. **cxxopts (jarro2783/cxxopts)**

Pros:

- Lightweight, header-only, C++11 compatible, very simple syntax.
- Widely used and well-maintained.
- Boost-like chained syntax; easy to integrate for small CLI tools.
- Required arguments enforcement supported.

Cons / Limitations:

- Nested subcommands are not natively supported; multi-level subcommand parsing requires manual dispatch.
- No per-command help aggregation.
- No tree introspection or automatic export.

Project Fit:

Suitable for simple CLIs with flags and basic commands. Not appropriate for a large, multi-level hierarchical CLI with documentation generation needs.

7. **p-ranav/argparse (argparse for Modern C++)**

Pros:

- C++17, single-header, clean API modeled after Python's argparse.

- Proper support for nested subcommands
- Per-command help works out-of-the-box; help is printed only for the requested subcommand.
- Actively maintained, MIT licensed, increasingly popular in modern C++ CLI comparisons.

Cons / Limitations:

- No native tree introspection — each subparser is independent; full command tree export requires manual bookkeeping.
- C++17 required
- No tree introspection or automatic export.

Project Fit:

Suitable for parsing and command structure itself; nested subcommands and per-command help are well supported. However, the lack of a pre-parse tree introspection API makes it less convenient for generating documentation automatically. Could be a good alternative if CLI11 were not available.

8. Sharg Parser (seqan/sharg-parser)

Pros:

- C++20, header-only, actively developed in bioinformatics ecosystem.
- Supports nested subcommands.
- Built-in documentation export with `--export-help` (HTML, man pages, etc.) - unique feature for auto-generating reference docs.

Cons / Limitations:

- Nested three-level commands require sequential calls - less elegant than CLI11.
- Exported help is reference-style; not suitable for narrative admin guides like Ceph's `radosgw-admin` docs.
- Requires C++20, GCC ≥ 11 or Clang ≥ 17 ; MSVC not supported.
- Smaller community and bioinformatics-specific origin may concern maintainers for general-purpose systems software.

Project Fit:

Interesting alternative for generating reference documentation directly from the binary. However, C++20 requirement, niche community, and limitations in narrative doc generation reduce its suitability as the primary framework.

9. Other frameworks briefly considered

- **docopt.cpp** – Very simple and intuitive, but lacks per-command help and tree introspection. Could be good for very small CLIs but not suitable for multi-level nested commands.
- **argh** – Minimalist and header-only; works for basic commands but lacks nested commands, per-command help, and tree export.
- **optparse / cpp-optparse** - Similar to Python's argparse, easy to understand, C++11 compatible. but lacks nested commands and tree introspection and has a limited community.
- **CLI** – Another lightweight option; supports simple parsing but does not scale well for nested subcommands and documentation needs.

Project Fit:

These alternatives were considered to ensure a broad survey, but none fully meet the project's requirements. They may be suitable for smaller utilities but are insufficient for a hierarchical, well-documented CLI.

Framework Selection Rationale

After evaluating multiple C++ CLI frameworks against the project's requirements, CLI11 is the only framework that fully satisfies all key needs:

- **Native support for nested subcommands:**
Supports multi-level command hierarchies out-of-the-box, without requiring manual dispatching logic.
- **Full tree introspection (critical requirement):**
Every command and subcommand is represented as a CLI::App object, with accessible APIs such as `get_subcommands()` and `get_options()`.
This enables recursive traversal of the entire command tree and straightforward export to JSON.

- **Direct support for documentation generation:**
The introspection capability allows the CLI structure to serve as a Single Source of Truth, feeding a Python-based pipeline that generates man pages automatically, and enables linking from the admin guide to relevant reference sections.
- **Auto-generated, context-aware help:**
Provides hierarchical --help output per command, with built-in validation features such as `require_subcommand()` that improve usability and error feedback.
- **Low integration overhead:**
Header-only framework (single CLI11.hpp file), actively maintained, and compatible with modern C++ standards used in Ceph (C++17).

Conclusion

- CLI11 is the recommended framework for this project, as it uniquely enables:
 - Multi-level command hierarchies
 - Hierarchical and user-friendly help output
 - Full command tree introspection for automated documentation generation
- Other frameworks fall short in one or more critical areas:
 - Lack of tree introspection
 - Limited or no support for deep subcommands
 - Inability to integrate cleanly with Sphinx-based documentation generation
- **Tradeoff:**
Introduces a new dependency, but this is minimal due to its header-only design and permissive BSD license.
- This makes it the only option that allows the C++ CLI binary to function as a true Single Source of Truth for both functionality and documentation.

4. Technical Solution & Proof of Concept

Approach

The solution replaces the current hand-written --help string and manually maintained documentation with a single pipeline: C++ definitions in CLI11 become the authoritative source for all outputs — runtime help, man page, and admin guide — with no separate files to maintain.

CLI11 definitions in C++

- --help / user --help (auto-generated at runtime)
- --export-tree → commands.json
 - generate_manpage.py → doc/man/8/radosgw-admin.rst
 - generate_adminguide.py → doc/radosgw/admin.rst (secondary goal)

Proof of Concept

PoC: [radosgw-admin-ux-docs-improvements/poc](https://github.com/ceph/radosgw-admin-ux-docs-improvements/poc)

I built a working PoC covering the full pipeline end to end. Full source, all demos with expected output, and step-by-step instructions (in the README.md):

Source files: radosgw_admin_poc.cpp, generate_manpage.py, generate_adminguide.py, README.md

Generated outputs files: commands.json, radosgw-admin.rst, admin.rst

The PoC demonstrates:

- Auto-generated --help with no hand-written string anywhere in the code
- Per-command and per-subcommand help scoped correctly to each level
- Three-level nested subcommands (bucket logging flush) with automatic hierarchical error listing
- Required argument enforcement with immediate clear error messages
- Two patterns for shared options: family helpers and flag-to-many-commands registration
- True global options (--conf, --cluster, --id) working at any nesting depth via recursive fallthrough()

- Option groups for related flags (demonstrated with rate-limit options)
- --export-tree producing a complete JSON command tree consumed by Python generators
- Man page RST generated automatically, matching the structure of the current doc/man/8/radosgw-admin.rst
- Admin guide skeleton generated with narrative preservation via a marker-based approach – human-written prose is never overwritten on regeneration. This is a secondary goal, with --help and man page taking priority.
- CI enforcement script that blocks PRs where the man page has not been regenerated after code changes. The design also supports generating documentation at publication time rather than committing generated files – to be decided with mentors.
- Production-quality validators: flag-like values rejected, duplicate flags rejected, std::optional for optional numeric arguments

5. Project Timeline (Estimated):

- **Community Bonding:**
 - Finalize the list of all radosgw-admin commands to be ported.
 - Identify which behavior changes are necessary for the migration and which can be introduced gradually via warnings.
 - Discuss the JSON schema for export with mentors.
 - Align on whether generated files will be committed or generated at publication time.
- **Weeks 1-3:**
 - Port a "Command Family" (e.g., user commands) to CLI11 while ensuring backward compatibility with existing syntax.
- **Weeks 4-7:**

- Iterative porting of all remaining commands (~200) and flags. Ensuring backward compatibility with existing command syntax.
- **Weeks 8-10:**
 - Developing the JSON export-tree mechanism and refining the Python RST generators for automated Man page updates.
 - If time permits: initial admin guide skeleton generation with narrative preservation, enabling future linking from the admin guide to relevant reference sections.
- **Weeks 11-12:**
 - Final integration into the Ceph build system, updating help.t tests to use the auto-generated output. Performing final documentation, end-to-end validation and cleanup.

6. Availability:

UTC+3. Available approximately 35 hours per week during GSoC. Generally offline from Friday at noon to Sunday morning, but responsive otherwise.